Qwen

Синтез Биологического Ума: Проектирование Событийно-Ориентированных Нейросетей на **Rust** для Эффективного Самообучения

Биологические Механизмы Энергоэффективного Обучения и Самоорганизации

Для создания высокоэффективных систем самообучения, превосходящих традиционные подходы, необходимо глубоко изучить и формализовать принципы, лежащие в основе биологических нейронных систем. Эти системы эволюционировали под давлением жестких ограничений по энергии и ресурсам, что сделало их чрезвычайно эффективными [12]. Анализ научных данных показывает, что ключевые механизмы, обеспечивающие эту эффективность, включают событийно-ориентированную вычислительную модель, локальные правила обучения, сложную систему нейромодуляции и способность к постоянному саморазвитию.

Основополагающим принципом является метаболическая эффективность. Мозг человека, составляя всего 2% массы тела, потребляет около 20% всей энергии в состоянии покоя [12]. Это создает мощный эволюционный импульс к минимизации затрат на любые процессы, включая обучение. В отличие от искусственных нейронных сетей (ИНС), где алгоритмы типа обратного распространения ошибки требуют глобальной оптимизации и многократных, дорогостоящих модификаций весов для каждого нового образца, что может приводить к "стиранию" ранее полученных знаний, биологический мозг использует децентрализованные, локальные правила обучения [11] [12]. Главным примером такого правила является временная зависимость потенциации синапса (STDP). STDP — это форма ассоциативной синаптической пластичности, при которой изменение силы связи между двумя нейронами зависит от точного временного порядка и интервала между их выбросами (спайками) [4] [5]. Классическое правило STDP предписывает, что если

предсинаптический нейрон спикнул до постсинаптического в течение короткого окна времени (обычно до ~20 мс), синапс укрепляется (долговременная потенциация, LTP). Если же порядок обратный, синапс ослабляется (долговременная депрессия, LTD) [5]. Этот механизм является локальным, поскольку он зависит только от информации, доступной непосредственно на самом синапсе (времени спайков двух соседних нейронов), что делает его идеальным для децентрализованных вычислений [11] [200]. Эта локальность позволяет сети учиться на локальных статистических корреляциях в данных, что значительно экономнее, чем глобальный анализ.

Однако простое STDP недостаточно для сложного поведения, такого как обучение с подкреплением, где последствия действий могут проявляться с задержкой. Эту проблему решает система нейромодуляции. Нейромодуляторы, такие как допамин, ацетилхолин, серотонин и норадреналин, действуют как глобальные сигналы, которые открывают или закрывают "ворота" для синаптической пластичности, индуцированной STDP [38] [129]. Они позволяют связывать события, произошедшие секунды назад, с положительным или отрицательным результатом. Допамин играет центральную роль, выступая в качестве сигнала предсказания вознаграждения [6] [10]. Когда полученный результат лучше ожидаемого, происходит выброс допамина, который "подтверждает" синаптические изменения, запомненные во время предшествующей активности [10]. Для этого используется трехфакторное правило обучения, известное как R-STDP (Reward-Modulated STDP), которое сочетает три компонента: 1) STDP для локального обучения (Hebbian); 2) краткосрочные "метки" (eligibility traces), которые хранят информацию о недавнем парном спайковом взаимодействии; 3) допаминовый сигнал, который консолидирует (делает постоянными) те изменения, которые были помечены, если они совпали со временем вознаграждения [143] [144] [194] [195]. Ацетилхолин также играет важную роль, модулируя чувствительность синапсов к STDP и контролируя скорость обучения (learning rate) [6] [9]. Например, совпадение пауз в активности холинергических нейронов с допаминовыми сигналами является мощным триггером для синаптической пластичности в стриатуме [69]. Теоретические модели связывают уровень ацетилхолина с величиной шага обучения, позволяя сети адаптироваться к уровню неопределенности в окружающей среде [6] [7].

Саморазвитие и постоянное обучение достигаются за счет механизмов структурной и функциональной пластичности. Одним из ключевых процессов

является синаптическая вырубка — удаление слабых или ненужных синаптических связей, которое происходит одновременно с усилением других [1]. Этот процесс driven both by spontaneous neural activity (например, "волны" в зрительной коре новорожденных) и опытом, полученным через органы чувств (например, при монокулярной депривации, когда один глаз закрывается, синапсы, связанные с ним, ослабляются) [1]. Синаптическая вырубка, в свою очередь, регулируется целым рядом факторов, включая активность нейронов, глиальные клетки (микроглия и астроциты) и даже молекулы иммунной системы, такие как компоненты комплементной системы C1q и C3, которые "метят" синапсы для удаления микроглией [1]. Параллельно, для поддержания стабильности сети в целом, работают механизмы гомеостатической пластичности. Они обеспечивают, чтобы общая активность нейронов оставалась в определенном диапазоне, предотвращая как взрывную активность (сеть "перегорает"), так и полное затухание [11]. Один из таких механизмов — адаптивный порог спайкования: если нейрон слишком часто реагирует на стимул, его порог возбуждения повышается, делая его менее чувствительным, и наоборот [11]. Еще одним интересным механизмом, объясняющим, как мозг может быстро учиться, не забывая старое, является синаптическое кеширование. Он предполагает разделение синаптической пластичности на две компоненты: быструю, но временную (transient), которая позволяет быстро адаптироваться к новым данным, и медленную, но долговечную (persistent), которая представляет собой консолидированные знания. Консолидация происходит периодически, когда накопленные временные изменения достигают определенного порога, что позволяет сохранить важные новые знания, не затрагивая старые [12]. Все эти процессы, от вычислений на уровне одного синапса до удаления целых связей, происходят в условиях жесткого энергетического бюджета, что делает биологические системы эталоном для создания следующего поколения эффективных вычислительных систем.

Биологический Принцип	Описание	Целевая Характеристика
Энергоэффективность	Мозг использует минимально возможное количество энергии для выполнения своих функций.	Минимальное потребление энергии, разреженные вычисления [12].
Локальное обучение (STDP)	Изменение силы синапса зависит исключительно от временной последовательности спайков двух соседних нейронов.	Децентрализованное управление, отсутствие необходимости в глобальной оптимизации [5] [11].
Нейромодуляция	Глобальные химические сигналы (допамин, ацетилхолин) модулируют пластичность и управляемость обучения.	Адаптивность, решение проблемы отложенного вознаграждения, управление эксплорацией/эксплуатацией [6] [10] [129].
Постоянное обучение	Система постоянно адаптируется, обучаясь на новых данных без стирания старых знаний.	Непрерывное обновление знаний, устойчивость к катастрофическому стиранию [109] [156].
Структурная пластичность	Процесс удаления слабых синапсов (вырубка) и формирования новых, более сильных связей.	Долгосрочная адаптация, специализация нейронных сетей, снижение "шума" [1].
Гомеостаз	Механизмы, поддерживающие стабильность общей активности нейронной сети.	Предотвращение взрывных или затухающих режимов работы, обеспечение стабильности [11].

От Биологии к Инженерии: Архитектуры Спайки-Нейронных Сетей (SNNs)

Прямой путь для формализации вышеописанных биологических принципов лежит через архитектуры, известные как спайки-нейронные сети (SNNs). В отличие от традиционных искусственных нейронных сетей (ANNs), которые используют непрерывные аналоговые значения для кодирования информации, SNNs стремятся воспроизвести более реалистичную модель обработки данных мозгом, где информация передается дискретными бинарными событиями — выбросами [85] [111]. Эта фундаментальная разница имеет далеко идущие последствия для вычислительной эффективности, способности к обучению и масштабируемости.

Ключевой особенностью SNNs является их событийно-ориентированная природа. Большинство нейронов в сети находятся в состоянии покоя и не участвуют в вычислениях в любой данный момент времени [13]. Вычисления происходят только тогда, когда нейрон генерирует выброс, который затем распространяется по сети и, возможно, вызовет выбросы в других нейронах [32] [33]. Такой подход, известный как event-driven computing, приводит к тому, что

вычислительная нагрузка становится разреженной и сильно зависит от входных данных [13]. Если в ANN каждый слой должен обработать все нейроны на каждом шаге, то в SNN вычисления концентрируются вокруг фактических событий, что приводит к значительному сокращению числа операций. Эта характеристика является прямым отражением энергоэффективности биологического мозга, где основной расход энергии происходит не на хранение информации, а на ее передачу в виде выбросов. Благодаря этой разреженности, SNNs потенциально в тысячи раз более энергоэффективны, особенно при их реализации на специализированном аппаратном обеспечении, известном как neuromorphic hardware [31 115159]. Однако даже в программной реализации на обычных процессорах event-driven подход уже дает преимущество в скорости и снижении использования памяти по сравнению с плотными матричными вычислениями, характерными для ANNs [19].

Центральным элементом многих SNN-архитектур является правило STDP, которое, как было показано, является естественным механизмом обучения для таких сетей [80 82]. STDP позволяет сети самостоятельно извлекать временные корреляции из входных данных, что делает его мощным инструментом для безучетного обучения [82]. Например, SNNs, обученные с помощью STDP, успешно справляются с задачами распознавания визуальных признаков, демонстрируя способность извлекать иерархические признаки из данных, аналогично многослойным ANN [80 81]. Кроме того, многие правила обучения на основе STDP являются локальными и итеративными, что идеально соответствует парадигме continual learning — способности модели учиться на новых задачах последовательно, не забывая предыдущие [109156]. Это контрастирует с большинством ANN, которые страдают от явления катастрофического стирания и требуют переобучения на всех данных, что делает их непригодными для автономных систем, работающих в постоянно меняющейся среде.

Реализация биологически плазматичных процессов в SNNs позволяет создавать более сложные и адаптивные модели. Например, встраивание систем нейромодуляции напрямую в архитектуру SNN позволяет реализовать трехфакторные правила обучения, такие как R-STDP, для решения задач обучения с подкреплением [143]. В этом случае нейронная сеть не просто учится на основе локальных правил, но и использует внешние сигналы (вознаграждение) для направления своего развития. Исследования показывают, что нейроморфные системы, сочетающие SNNs и нейромодуляцию, могут эффективно решать задачи непрерывного обучения, адаптируясь к изменяющимся условиям среды [109156]. Кроме того, возможность моделировать

структурную пластичность, такую как синаптическая вырубка, позволяет создавать архитектуры, которые могут изменять свою собственную топологию, удаляя ненужные связи и становясь более специализированными и эффективными с течением времени [105][106]. Например, модель первичной зрительной коры, построенная на основе LIF-нейронов и STDP, смогла самоорганизоваться для создания Gabor-like рецептивных полей, которые очень похожи на рецептивные поля реальных нейронов в зрительной коре кошки и макаки, демонстрируя высокую степень биологической достоверности и эффективности [11]. Таким образом, SNNs представляют собой не просто альтернативу ANN, а целостную вычислительную парадигму, которая органично интегрирует принципы энергоэффективности, локального обучения, нейромодуляции и структурной пластичности, предлагая уникальную платформу для создания истинно саморазвивающихся и ресурсно-минимальных ИИ-систем.

Технологический Стек **Rust** для Реализации Нейробиологически-Платных Моделей

Выбор языка программирования для реализации сложных и требовательных к ресурсам биологически-вдохновленных моделей является критически важным. Язык Rust, благодаря своей философии и набору инструментов, предлагает уникальные возможности, которые делают его привлекательной альтернативой Python-экосистеме, доминирующей в машинном обучении. Для создания высокопроизводительных систем, подобных тем, что обсуждались ранее, Rust предоставляет зрелую и развивающуюся экосистему библиотек, а также фундаментальные свойства, направленные на максимальную эффективность.

Экосистема Rust для машинного обучения, особенно в области SNNs, активно развивается и включает в себя ряд специализированных библиотек. Эти библиотеки предоставляют необходимые строительные блоки для моделирования биологически-правдоподобных нейронных сетей.

- `neuromod`: Эта библиотека позиционируется как биологически-обоснованный движок для SNNs, который предоставляет гибкие строительные блоки, такие как различные модели нейронов (например, ЛИФ, Izhikevich), классические синаптические правила (например, STDP) и гибкие механизмы нейромодуляции [87][186]. Её ориентация на биологическую

достоверность делает её идеальным кандидатом для реализации исследуемых в данном отчете концепций.
- `spiking_neural_networks`: Это еще одна библиотека, сфокусированная на моделировании динамики нейронов, включая их neurotransmission (передачу нейромедиаторов) и расчет взаимодействий между ними во времени [18] [20]. Она предоставляет базовые инструменты для построения и симуляции сетей.
- `omega_snn`: Данная библиотека используется в проекте ExoGenesis Omega и предназначена для создания "мозгоподобных" динамических слоев, что указывает на её применимость для создания сложных, динамических моделей, вдохновленных мозгом [128].
- `neural-rs` и `fomorians/neural-rs`: Это более общие библиотеки для построения нейронных сетей на Rust, некоторые из которых имеют акцент на SNNs [183][184]. Хотя они могут быть менее специализированными, они демонстрируют жизнеспособность и популярность данной области.
- `burn`: Это амбициозный проект, стремящийся стать полноценным фреймворком для глубокого обучения на Rust, сопоставимым с PyTorch или TensorFlow [47] [73]. Важно отметить, что `burn` полностью написан на Rust и не зависит от внешних C++ бэкендов, что позволяет для глубокой интеграции с типовой системой Rust [77]. Он может служить мощным тензорным движком для оптимизации вычислений внутри пользовательских, более специализированных SNN-моделей.

Существование этих инструментов подтверждает, что Rust уже является рабочей платформой для исследований в области SNNs. Более того, имеются примеры их применения в реальных задачах, например, для создания системы фильтрации flash-crash'ов в DeFi рынках, что свидетельствует о надежности и производительности платформы для практических применений [169]. Значительный список библиотек и проектов, связанных с машинным обучением в Rust, поддерживается сообществом, что говорит о растущем интересе и инвестициях в развитие этой экосистемы [62] [180].

Помимо специализированных библиотек, ключевыми преимуществами Rust для данной задачи являются его фундаментальные свойства. Zero-Cost Abstractions гарантируют, что написание высокоуровневого, безопасного и удобного кода не приводит к накладным расходам в момент выполнения [21] [22]. Это позволяет разработчикам реализовывать сложные биологические концепции, такие как объекты нейронов, синапсов и нейромодуляторов, в виде четко структурированных структур и трейтов, которые будут скомпилировы в

максимально быстрый исполняемый код, сравнимый с ручной реализацией на C/C++ [57] [59] [64]. Безопасность памяти, обеспечиваемая системой владения и ссылок без сборщика мусора, исключает целый класс ошибок, присущих C++, и обеспечивает предсказуемое время выполнения, что критически важно для систем реального времени [192]. Наконец, возможность тонкой настройки расположения данных в памяти и обработки данных без копирования позволяет добиться максимальной производительности на современных архитектурах процессоров, что будет подробно рассмотрено в следующем разделе. Все эти факторы в совокупности делают Rust мощной и перспективной платформой для реализации следующего поколения высокоэффективных, биологически-вдохновленных вычислительных систем.

Достижение Высокой Производительности: Оптимизация на **Rust**

Идея создания системы самообучения на Rust, превосходящей по скорости и эффективности PyTorch, основывается не просто на выборе другого языка, а на использовании уникального набора его характеристик, которые позволяют достичь максимальной производительности на аппаратном уровне. Хотя PyTorch, будучи написанным на C++, является чрезвычайно быстрым фреймворком, его Python-интерфейс и динамическая природа вносят накладные расходы, которые Rust, компилирующийся в нативный машинный код, изначально лишён [54] [55]. Производительность Rust достигается за счет трех ключевых столпов: нулевых абстракций, эффективного управления памятью и контроля над кэш-локальностью.

Первый и самый важный принцип — это нулевые абстракции. В Rust любая конструкция языка, которая кажется высокоуровневой (например, использование векторов, структур данных, трейтов для полиморфизма), компилируется в машинный код, который не уступает по производительности вручную оптимизированной низкоуровневой реализации [21] [22]. Компилятор Rust, используя мощный инструментарий LLVM, выполняет множество оптимизаций на этапе компиляции, таких как встраивание функций, раскрутка циклов и другие преобразования, которые делают код чрезвычайно быстрым [116] [178]. Это означает, что разработчик может писать код, который легко читать, поддерживать и проверять, не беспокоясь о том, что он будет медленнее, чем

если бы он писал его на C. Для задачи моделирования SNNs это огромное преимущество: можно создать богатую объектную модель, где есть отдельные классы для нейронов, синапсов, нейромодуляторов и правил обучения, и быть уверенным, что в итоге получится исполняемый файл, работающий с минимальными накладными расходами [59] [118]. Это позволяет сосредоточиться на корректности и элегантности архитектуры, не жертвуя при этом производительностью.

Второй столп — это безопасность памяти и эффективное управление данными. В отличие от Python, где данные часто копируются между различными частями системы (например, между CPU и GPU), Rust стремится к работе с данными без копирования. Это достигается через систему владения и ссылок, которая на этапе компиляции гарантирует отсутствие ошибок доступа к памяти, таких как "зависшие указатели" или "двойное освобождение". В контексте машинного обучения это критически важно, поскольку перемещение больших объемов данных (весов, входных тензоров) является одним из главных узких мест в производительности, особенно при использовании графических процессоров [41]. Библиотеки в экосистеме Rust, такие как `tinyml-rs`, уже демонстрируют применение zero-copy сериализации для загрузки весов моделей, что позволяет избежать дополнительных затрат на их загрузку в память [42] [43] [44]. Это позволяет создавать системы, которые максимально эффективно используют доступную память и минимизируют задержки, связанные с ее доступом.

Третий столп — это контроль над расположением данных в памяти и, как следствие, кэш-локальностью. Современные процессоры крайне чувствительны к тому, как данные расположены в памяти. Доступ к данным, находящимся в одном кэш-ряду, на несколько порядков быстрее, чем к данным, распределенным по разным строкам [97] [98]. Rust дает разработчику полный контроль над этим. Можно выбрать между стандартным расположением "Array of Structures" (AoS), где данные о каждом объекте хранятся вместе, и "Structure of Arrays" (SoA), где все значения первого поля хранятся в одном массиве, все значения второго поля — в другом и так далее [101] [133]. Для многих задач, особенно для векторных операций и SIMD-инструкций, SoA является значительно более эффективным, так как он обеспечивает высокую пространственную локальность доступа к данным [99] [102]. В контексте SNN, где необходимо одновременно обновлять состояния тысяч нейронов или синапсов, переход на SoA-расположение для массивов состояний (например, `voltages`, `spike_times`, `weights`) может кардинально снизить количество промахов в кэше и ускорить внутренние вычислительные циклы [102]. Эта возможность

тонкой настройки является труднодостижимой в высокоуровневых языках, таких как Python, и является одной из причин, по которой Rust может обеспечить превосходство в производительности на задачах, чувствительных к архитектуре памяти.

Хотя существуют противоречивые данные о производительности, сопоставление Rust-фреймворков с PyTorch показывает, что Rust имеет значительный потенциал. Некоторые тесты показывают, что на GPU Rust-решения могут быть медленнее из-за меньшей зрелости GPU-бэкендов [75]. Однако другие исследования и практические примеры демонстрируют, что на CPU Rust может быть значительно быстрее, а также потреблять меньше памяти, особенно в сценариях инференса [76] [89]. Наибольший успех Rust в ML был достигнут именно в области инференса, где он заменяет Python и C++ для создания производительных и безопасных серверов для обслуживания моделей [127] [179] [192]. Создание собственного фреймворка для обучения, подобного `burn`, остается нетривиальной задачей, но полностью реализуемой благодаря вышеперечисленным преимуществам Rust [47] [73].

Стратегия Разработки и Тестирования: Практическое Руководство

На основе проведенного анализа можно сформулировать комплексную стратегию для создания высокоэффективной, ресурсно-минимальной системы самообучения на Rust, вдохновленной биологическими принципами. Эта стратегия охватывает выбор архитектуры, технологического стека, этапы разработки и методы тестирования, направленные на достижение поставленной цели.

Первоначальный шаг — это определение концептуальной архитектуры. Наиболее перспективным подходом является использование **Spiking Neural Network (SNN)** в качестве основы. SNNs естественным образом воплощают принципы событийно-ориентированной вычислительной модели, характерной для биологического мозга [13] [32]. Обучение должно быть основано на **Spike-Timing-Dependent Plasticity (STDP)** как на основном локальном правиле, позволяющем сети извлекать временные корреляции из данных [5] [80]. Однако для достижения сложного поведения, такого как обучение с подкреплением,

необходимо внедрить систему нейромодуляции. Архитектура должна включать хотя бы два ключевых нейромодулятора: 1. Допамин: Реализация должна поддерживать **Reward-Modulated STDP (R-STDP)**. Это потребует добавления механизма "eligibility traces" — краткосрочных "меток", которые хранят информацию о синаптических событиях, чтобы их можно было консолидировать позже под действием допаминового сигнала [144][194][195]. 2. Ацетилхолин: Необходимо реализовать его роль в модуляции пластичности и контроле скорости обучения (learning rate), что согласуется с теоретическими моделями, связывающими уровень ACh с величиной шага обучения [6] [7].

Для обеспечения "саморазвития" и постоянного обучения, архитектура должна быть дополнена механизмами структурной и функциональной пластичности. Это включает синаптическую вырубку (удаление слабых связей) и гомеостатическую стабилизацию, например, через адаптивный порог спайкования для каждого нейрона, чтобы предотвратить взрывную или затухающую активность сети [1] [11]. Для эффективного continual learning следует рассмотреть реализацию принципа синаптического кеширования, разделяющего пластичность на быструю, но временную (cache) и медленную, но долговечную (consolidated weights) [12].

Технологический стек для реализации должен быть выбран с учетом биологической достоверности и производительности. Рекомендуется начать с библиотек, сфокусированных на биологической правдоподобности, таких как `neuromod` [87][186] или `omega_snn` [128]. Эти библиотеки уже предоставляют базовые строительные блоки, такие как модели нейронов (например, Leaky Integrate-and-Fire) и правила обучения (STDP), что позволит сосредоточиться на интеграции более сложных механизмов, таких как нейромодуляция. Альтернативно, можно использовать `burn` [73] как мощный тензорный движок, который может обеспечить высокопроизводительные базовые операции (например, матричные умножения, свертки), в то время как логика SNN и правил обучения будет реализована поверх него в виде пользовательских слоев.

Процесс разработки можно разделить на несколько итерационных этапов: 1. Создание базового прототипа: Реализовать простую SNN (например, из LIF-нейронов) со стандартным STDP. Цель этого этапа — убедиться, что базовая модель работает корректно и способна выучивать простые временные шаблоны в сгенерированных или реальных спайковых данных. 2. Интеграция нейромодуляции: Добавить в модель механизм R-STDP с допаминовым сигналом. Создать простую среду для обучения с подкреплением (например,

задачу о двух-armed bandit или простой MDP) и проверить, что агент способен научиться выполнять действия для получения вознаграждения, а допаминовая сигнализация корректно влияет на процесс обучения. 3. Реализация механизмов саморазвития: Внедрить гомеостатические механизмы (например, адаптивный порог) и, возможно, простой механизм синаптической вырубки. Провести эксперименты, демонстрирующие способность сети поддерживать стабильность и адаптироваться к изменяющимся статистическим свойствам входных данных. 4. Оптимизация на **Rust**: После того как работающая архитектура будет создана, следует сосредоточиться на оптимизации. Это включает использование нулевых абстракций для написания чистого кода, переход на Structure of Arrays (SoA) для массивов состояний нейронов и синапсов для повышения кэш-локальности [102], и реализацию обработки данных без копирования для минимизации накладных расходов [42]. 5. Тестирование и сравнение: Провести всестороннее тестирование производительности. Сравнить разработанную систему с эталонными решениями (например, ANN/SNN-эмюлятор на PyTorch) по нескольким ключевым метрикам: скорость обучения и вывода (inference), потребление памяти и энергии (если возможно), а также качество решения задачи (например, итоговое вознаграждение в задачах обучения с подкреплением).

Важно признать ограничения данного подхода. Биологические системы невероятно сложны, и наша модель будет являться их упрощением. Достижение производительности, сопоставимой с highly optimized CUDA-ядрами PyTorch/TensorFlow, на GPU остается нетривиальной задачей, требующей значительных усилий в разработке GPU-бэкенда для Rust. Тем не менее, предложенная стратегия, основанная на синтезе биологических принципов и использовании мощных инструментов Rust, представляет собой прямой и обоснованный путь к созданию следующего поколения высокоэффективных, ресурсно-минимальных и саморазвивающихся ИИ-систем.

Справка

1. Mechanisms governing activity-dependent synaptic pruning in ... https://pmc.ncbi.nlm.nih.gov/articles/PMC8541743/

2. Activity-Dependent Dendritic Spine Structural Plasticity Is ... https://www.sciencedirect.com/science/article/pii/S0896627305008871
3. Review Curiosity-driven exploration: foundations in neuroscience and ... https://www.sciencedirect.com/science/article/pii/S0166223623002400
4. Timing to be precise? An overview of spike timing ... https://www.nature.com/articles/s41380-023-02027-w
5. The spike timing dependence of plasticity - PMC - NIH https://pmc.ncbi.nlm.nih.gov/articles/PMC3431193/
6. Lifelong Reinforcement Learning via Neuromodulation - arXiv https://arxiv.org/html/2408.08446v1
7. (PDF) Lifelong Reinforcement Learning via Neuromodulation https://www.researchgate.net/publication/383217156_Lifelong_Reinforcement_Learning_via_Neuromodulation
8. The findings of a study in Nature Neuroscience suggest that ... https://www.facebook.com/NaturePortfolioJournals/posts/the-findings-of-a-study-in-nature-neuroscience-suggest-that-cholinergic-dynamics/1425389539617308/
9. Timing of acetylcholine release determines if dopamine promotes ... https://www.news-medical.net/news/20260325/Timing-of-acetylcholine-release-determines-if-dopamine-promotes-movement-or-learning.aspx
10. Dopamine: The Neuromodulator of Long-Term Synaptic Plasticity ... https://www.mdpi.com/2073-4409/10/4/735
11. Emergence of sparse coding, balance and decorrelation from ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12716757/
12. Energy efficient synaptic plasticity https://elifesciences.org/articles/50804
13. Rust Implementation Plan for a 'Super-Turing' Spiking AI ... https://gist.github.com/ruvnet/ec75041bf3cf1c6ce8c18fc0ba1d967d
14. Neuromorphic Spiking Neural Networks and Their Memristor ... https://pmc.ncbi.nlm.nih.gov/articles/PMC6747825/
15. Rust SNNs https://open-neuromorphic.org/neuromorphic-computing/software/snn-frameworks/rust-snns/
16. Using a neural network implementation as a teaching tool ... https://users.rust-lang.org/t/using-a-neural-network-implementation-as-a-teaching-tool-for-intermediate-rust-feedback-welcome/138074
17. I Added CNNs and GPU support to my Neural Network ... https://www.reddit.com/r/rust/comments/14qv5mx/i_added_cnns_and_gpu_support_to_my_neural_network/
18. spiking_neural_networks - crates.io: Rust Package Registry https://crates.io/crates/spiking_neural_networks

19. Spiffy: Efficient Implementation of CoLaNET for Raspberry Pi https://arxiv.org/html/2506.18306v1
20. spiking_neural_networks - Rust https://docs.rs/spiking_neural_networks/latest/spiking_neural_networks/
21. What specifically are all the zero-cost abstractions in Rust? - Reddit https://www.reddit.com/r/rust/comments/bo13qq/what_specifically_are_all_the_zerocost/
22. Zero-Cost Abstractions in Rust: Power Without the Price - DockYard https://dockyard.com/blog/2025/04/15/zero-cost-abstractions-in-rust-power-without-the-price
23. Rust has zero cost abstraction.What does this mean in a practical sense https://users.rust-lang.org/t/rust-has-zero-cost-abstraction-what-does-this-mean-in-a-practical-sense/100556
24. Learning Rust: Understanding Zero-Cost Abstraction with Filter ... https://ranveersequeira.medium.com/learning-rust-understanding-zero-cost-abstraction-with-filter-and-map-e967d09fff79
25. Zero Cost Abstractions - Rust - YouTube https://www.youtube.com/watch?v=UZFgN-t0Fgc
26. What does 'Zero Cost Abstraction' mean? - Stack Overflow https://stackoverflow.com/questions/69178380/what-does-zero-cost-abstraction-mean
27. Zero-Cost Abstractions in Rust: Asynchronous Programming ... https://dev.to/pranta/zero-cost-abstractions-in-rust-asynchronous-programming-without-breaking-a-sweat-221b
28. Zero Cost Abstractions - Without boats https://without.boats/blog/zero-cost-abstractions/
29. Learning Rust: Zero Cost Abstraction | by Adrian Macal - Level Up Coding https://levelup.gitconnected.com/learning-rust-zero-cost-abstraction-8079904d993a
30. Rust zero-cost abstractions vs. SIMD - Lobsters https://lobste.rs/s/2gwqlh/rust_zero_cost_abstractions_vs_simd
31. SpikingJelly: An open-source machine learning infrastructure platform for ... https://pmc.ncbi.nlm.nih.gov/articles/PMC10558124/
32. Bio-Inspired Neural Network Dynamics-Aware Reinforcement ... https://www.mdpi.com/2313-7673/11/1/47
33. Event-Driven Processing and Learning with Spiking Neural ... https://sites.northwestern.edu/ivpl/event-driven-processing-and-learning-with-spiking-neural-networks/
34. Spyx: A Library for Just-In-Time Compiled Optimization of Spiking ... https://arxiv.org/html/2402.18994v1

35. Timing is not Everything: Neuromodulation Opens the STDP ... https://pmc.ncbi.nlm.nih.gov/articles/PMC3059689/
36. Dopaminergic Neuromodulation of Spike Timing Dependent Plasticity ... https://www.frontiersin.org/journals/cellular-neuroscience/articles/10.3389/fncel.2021.668980/full
37. Sequential neuromodulation of Hebbian plasticity offers ... - eLife https://elifesciences.org/articles/27756
38. Neuromodulation of Spike-Timing-Dependent Plasticity https://www.researchgate.net/publication/335315786_Neuromodulation_of_Spike-Timing-Dependent_Plasticity_Past_Present_and_Future
39. [PDF] Druart, et al. 2026 Main Text Title - bioRxiv https://www.biorxiv.org/content/10.64898/2026.08.05.742858.full.pdf
40. Timing is not everything: Neuromodulation opens the STDP ... https://pure.johnshopkins.edu/en/publications/timing-is-not-everything-neuromodulation-opens-the-stdp-gate-3/
41. Rust for AI Compute - Typedef https://www.typedef.ai/resources/rust-based-ai-compute-trends
42. CanReader/tinyml-rs - GitHub https://github.com/CanReader/tinyml-rs
43. Zero-Copy All the Things! (Zero-Copy #2) - In Pursuit of Laziness http://manishearth.github.io/blog/2022/08/03/zero-copy-2-zero-copy-all-the-things/
44. Zero-Copy Data Serialization in Rust for High-Throughput ... - LinkedIn https://www.linkedin.com/posts/martinuke0_implementing-zero-copy-data-serialization-activity-7460102417183936512-O3N6
45. Designing state machines in Rust - DEV Community https://dev.to/senyeezus/designing-state-machines-in-rust-252k
46. GT-RIPL/Continual-Learning-Benchmark https://github.com/GT-RIPL/Continual-Learning-Benchmark
47. Burn vs. Candle: The Realistic Benchmark Battle for Rust's ... https://bytemedaily.medium.com/burn-vs-candle-the-realistic-benchmark-battle-for-rusts-machine-learning-ecosystem-7ee7618ef715
48. Towards Biologically Plausible Computing https://arxiv.org/html/2406.16062v1
49. Biologically-Plausible Continual Learning https://www.cs.manchester.ac.uk/study/postgraduate-research/research-projects/description/?projectid=22461
50. Training biologically plausible recurrent neural networks ... https://proceedings.neurips.cc/paper_files/paper/2023/file/65ccdfe02045fa0b823c5fa7ffd56b66-Paper-Conference.pdf

51. Current Deep Learning Ecosystem from a Rust Developer ... https://tiberiusferreira.github.io/blog/posts/current_deep_learning_ecosystem_from_a_rust_developer_perspective/
52. Continual Learning papers list, curated by ContinualAI · GitHub https://github.com/ContinualAI/continual-learning-papers
53. Benchmark Burn versus PyTorch on one million parameters ... - Instagram https://www.instagram.com/reel/DYsd4HkjyfB/
54. Rust vs PyTorch Speed - YouTube https://m.youtube.com/shorts/BcwxKVyQxQw
55. Python vs. Rust for Neural Networks - Hacker News https://news.ycombinator.com/item?id=20728288
56. Python and Rust in AI: A Pragmatic Path Forward - FRANKI T https://www.francescatabor.com/articles/2025/11/25/python-and-rust-in-ai-a-pragmatic-path-forward
57. Zero-Cost Abstractions in Rust - Unlocking High ... https://www.reddit.com/r/rust/comments/12ejwxe/zerocost_abstractions_in_rust_unlocking_high/
58. Truly Zero Cost - Vorner's random stuff - GitHub Pages https://vorner.github.io/2018/11/11/truly-zero-cost.html
59. Tutorial: How to Build a Simple Neural Network in Rust - Medium https://hietalajulius.medium.com/tutorial-how-to-build-a-simple-neural-network-in-rust-1c63fb7439f6
60. Rust zero-cost abstractions vs. SIMD https://turbopuffer.com/blog/zero-cost
61. The Rust ML Ecosystem Is Quietly Becoming Production-Ready https://medium.com/@theopinionatedev/the-rust-ml-ecosystem-is-quietly-becoming-production-ready-f4d348bea118
62. e-tornike/best-of-ml-rust: A ranked list of awesome machine learning ... https://github.com/e-tornike/best-of-ml-rust
63. The Beginner's Guide to Machine Learning with Rust https://machinelearningmastery.com/the-beginners-guide-to-machine-learning-with-rust/
64. Rust Meets AI: Why the Future of Machine Learning Runs on Safe ... https://levelup.gitconnected.com/rust-meets-ai-why-the-future-of-machine-learning-runs-on-safe-fast-code-ce742095e07e
65. Rust Ecosystem for AI & LLMs - HackMD https://hackmd.io/@Hamze/Hy5LiRV1gg
66. Neuromodulation of Hippocampal-Prefrontal Cortical Synaptic ... https://www.frontiersin.org/journals/cellular-neuroscience/articles/10.3389/fncel.2021.732360/full
67. Neuromodulation of Hippocampal-Prefrontal Cortical Synaptic ... https://pmc.ncbi.nlm.nih.gov/articles/PMC8542677/

68. The Cellular and Synaptic Actions of Dopamine During Behavior https://www.biorxiv.org/content/10.64898/2026.08.05.742858v1.full
69. Intrinsic dopamine and acetylcholine dynamics in the striatum of mice https://pmc.ncbi.nlm.nih.gov/articles/PMC11577287/
70. [PDF] Neuromorphic Computing with Large Scale Spiking Neural Networks https://www.preprints.org/manuscript/202503.1505/download/final_file
71. neuromod - Rust - Docs.rs https://docs.rs/neuromod
72. Advancements in neuromorphic computing for bio-inspired artificial ... https://www.sciencedirect.com/science/article/pii/S0925231225018934
73. GitHub - tracel-ai/burn: Burn is a next generation tensor library and Deep ... https://github.com/tracel-ai/burn
74. Direct Training High-Performance Deep Spiking Neural Networks - arXiv https://arxiv.org/html/2405.04289v2
75. Rust Deep Learning Framework 15% Slower than PyTorch - LinkedIn https://www.linkedin.com/posts/bereader_hand-written-cuda-vs-pytorch-honest-benchmarks-activity-7493245574557786112-RbGS
76. Reduced Memory Usage: Burn's Rusty Approach to Tensor Handling https://burn.dev/blog/burn-rusty-approach-to-tensor-handling/
77. Chapter 4 - Deep Learning via Rust - RantAI https://dlvr.rantai.dev/docs/part-i/chapter-4/
78. Burn: PyTorch Integration for Deep Learning https://rage.pythai.net/burn-pytorch-integration-for-deep-learning/
79. A neuromorphic implementation of multiple spike-timing ... https://pmc.ncbi.nlm.nih.gov/articles/PMC4438254/
80. STDP-based spiking deep convolutional neural networks ... https://www.sciencedirect.com/science/article/abs/pii/S0893608017302903
81. SSTDP: Supervised Spike Timing Dependent Plasticity for ... https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2021.756876/full
82. Deep Unsupervised Learning Using Spike-Timing ... https://arxiv.org/html/2307.04054v2
83. A Spike Neural Network Model for Lateral Suppression of ... https://www.mdpi.com/2076-3417/12/12/5980
84. Stable Spike-Timing Dependent Plasticity Rule for Multilayer ... https://ywang393.expressions.syr.edu/wp-content/uploads/2016/07/Stable-spike-timing-dependent-plasticity-rule-for-multilayer-unsupervised-and-supervised-learning.pdf
85. Contemporary implementations of spiking bio-inspired ... https://arxiv.org/html/2412.17926v1

86. torsh_optim::neuromorphic - Rust https://docs.rs/torsh-optim/latest/torsh_optim/neuromorphic/index.html
87. Neuromod https://lib.rs/crates/neuromod
88. Spike-based neuromorphic computing: An overview from ... https://research.tue.nl/en/publications/spike-based-neuromorphic-computing-an-overview-from-bio-inspirati/
89. Taking ML to production with Rust: a 25x speedup https://www.lpalmieri.com/posts/2019-12-01-taking-ml-to-production-with-rust-a-25x-speedup/
90. Cholinergic modulation of striatal microcircuits - PMC - NIH https://pmc.ncbi.nlm.nih.gov/articles/PMC6587740/
91. Spike-timing dependent plasticity in the striatum - Frontiers https://www.frontiersin.org/journals/synaptic-neuroscience/articles/10.3389/fnsyn.2010.00006/full
92. Cholinergic-dependent dopamine signals in mouse dorsal striatum ... https://www.biorxiv.org/content/10.1101/2025.09.30.679538v1.full-text
93. Spike-Timing Dependent Plasticity in the Striatum - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC3059675/
94. Striatal Cholinergic Interneurons Modulate Spike-Timing in ... https://dspace.mit.edu/entities/publication/ef208727-dce9-445b-bd99-bde3698fc769
95. State-dependent cortical processing: Cholinergic modulation of ... https://escholarship.org/uc/item/8j96f8hg
96. Dopamine regulates synchronicity in the activity of striatal neurons https://medicalxpress.com/news/2020-11-dopamine-synchronicity-striatal-neurons.html
97. locality, access, allocate, cache lines - tutorials https://users.rust-lang.org/t/tutorial-on-rust-memory-performance-latency-locality-access-allocate-cache-lines/130718
98. Inner loops are faster if they don't span a cache line : r/rust https://www.reddit.com/r/rust/comments/fcjx43/inner_loops_are_faster_if_they_dont_span_a_cache/
99. Thinking in Data Layout, Not Objects https://medium.com/@michaelstebel/thinking-in-data-layout-not-objects-bfef321e083d
100. Build with Naz : Rust, Memory performance & latency http://developerlife.com/2025/05/19/rust-mem-latency/
101. Is my understanding of AoS vs SoA advantages ... https://stackoverflow.com/questions/40163722/is-my-understanding-of-aos-vs-soa-advantages-disadvantages-correct
102. kizzasi_model::cache_friendly - Rust https://docs.rs/kizzasi-model/latest/kizzasi_model/cache_friendly/index.html

103. Cache Optimization in Rust: From HashMap Surprises to ... https://dev.to/lvc1d/cache-optimization-in-rust-from-hashmap-surprises-to-4x-image-processing-speedup-256l
104. Efficient Spike-Driven Learning With Dendritic Event-Based ... https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2021.601109/full
105. GLSNN: A Multi-Layer Spiking Neural Network Based ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC7689090/
106. State Transition of Dendritic Spines Improves Learning of ... https://proceedings.mlr.press/v162/chen22ac/chen22ac.pdf
107. Training Spiking Neural Networks with Event-driven ... https://proceedings.neurips.cc/paper_files/paper/2022/hash/c4e5f4de1b3cfc838eec6484d0b85378-Abstract-Conference.html
108. Compression and Inference of Spiking Neural Networks on ... https://arxiv.org/html/2511.12136
109. A highly energy-efficient multi-core neuromorphic ... https://www.nature.com/articles/s41467-026-70586-x
110. Optimizing the Energy Consumption of Spiking Neural ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC7339957/
111. Exploring Neuromorphic Computing Based on Spiking ... https://dl.acm.org/doi/full/10.1145/3571155
112. Benchmarking the performance of neuromorphic and ... https://impact.ornl.gov/en/publications/benchmarking-the-performance-of-neuromorphic-and-spiking-neural-n/
113. Benchmarking Neuromorphic Computing for Inference https://yoda.csem.ch/bitstream/handle/20.500.12839/1057/Benchmarking_Neuromorphic_Computing_for_Inference.pdf?sequence=1&isAllowed=y
114. Benchmarking the Performance of Neuromorphic and ... https://www.osti.gov/servlets/purl/1779135
115. Exploiting deep learning accelerators for neuromorphic ... https://iopscience.iop.org/article/10.1088/2634-4386/ad2373/pdf
116. Understanding Rust's Zero-Cost Abstractions - Reintech https://reintech.io/blog/understanding-rust-zero-cost-abstractions
117. How to understand zero cost abstraction for Rust - tutorials https://users.rust-lang.org/t/how-to-understand-zero-cost-abstraction-for-rust/9761
118. How Rust Gives You High-Level Code Without the Overhead - Medium https://medium.com/@adamszpilewicz/zero-cost-abstractions-in-rust-how-rust-gives-you-high-level-code-without-the-overhead-1b2b1908f591

119. Zero Cost Abstractions in C++20, Rust, & Zig - YouTube https://www.youtube.com/watch?v=43X9ia-qpds
120. Bonus Tutorial: Spike-timing dependent plasticity (STDP) https://compneuro.neuromatch.io/tutorials/W2D4_BiologicalNeuronModels/student/W2D4_Tutorial4.html
121. Zero Cost Abstractions | Hacker News https://news.ycombinator.com/item?id=19931537
122. SSTDP: Supervised Spike Timing Dependent Plasticity for Efficient ... https://pmc.ncbi.nlm.nih.gov/articles/PMC8603828/
123. Polysynaptic inhibition between striatal cholinergic interneurons ... https://www.nature.com/articles/s41467-020-18882-y
124. Cholinergic interneuron inhibition potentiates corticostriatal ... https://www.cell.com/cell-reports/fulltext/S2211-1247(22)00828-2
125. Synaptic-like axo-axonal transmission from striatal cholinergic ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9509469/
126. Rust for AI/ML in 2026 — Can It Replace Python? (Burn ... - YouTube https://www.youtube.com/watch?v=4VNcmIHvKGA
127. Efficient and Cross-Platform AI Inference Apps Using Rust and Wasm https://www.youtube.com/watch?v=HI2w_Qw06YE
128. omega_snn - Rust https://docs.rs/omega-snn
129. Neuromodulated Spike-Timing-Dependent Plasticity, and ... https://www.frontiersin.org/journals/neural-circuits/articles/10.3389/fncir.2015.00085/full
130. Biologically Plausible Dopamine-Modulated STDP Model ... https://www.biorxiv.org/content/10.64898/2026.07.17.739276v1.full.pdf
131. The Memory Layout Trick That Turns Rust Structs Into ... https://medium.com/@theopinionatedev/the-memory-layout-trick-that-turns-rust-structs-into-cache-miss-monsters-963a3a72d1f3
132. Align struct to cache lines in Rust https://stackoverflow.com/questions/41090078/align-struct-to-cache-lines-in-rust
133. tim-harding/soa-rs: An SoA library for Rust https://github.com/tim-harding/soa-rs
134. State Transition of Dendritic Spines Improves Learning of ... https://proceedings.mlr.press/v162/chen22ac.html
135. Mapping high-performance RNNs to in-memory ... https://arxiv.org/html/1905.10692v4
136. trustformers_core::neuromorphic - Rust https://docs.rs/trustformers-core/latest/trustformers_core/neuromorphic/index.html

137. Rust Generics and Monomorphization: Zero-Cost ... https://medium.com/@adamszpilewicz/rust-generics-and-monomorphization-zero-cost-abstractions-in-action-d6d2252d6f88

138. Will neuromorphic chips become the definitive solution to ... https://www.reddit.com/r/LocalLLaMA/comments/1sdfuu1/will_neuromorphic_chips_become_the_definitive/

139. Benchmarking Neuromorphic Computing for Inference https://www.riverpublishers.com/downloadchapter.php?file=RP_9788770227902C1.pdf

140. Energy Efficiency in Spiking vs Traditional Neural Networks https://eureka.patsnap.com/report-energy-efficiency-in-spiking-vs-traditional-neural-networks

141. Are we learning yet? https://www.arewelearningyet.com/

142. Neuromodulated Spike-Timing-Dependent Plasticity ... https://pmc.ncbi.nlm.nih.gov/articles/PMC4717313/

143. Reward-Modulated STDP in Spiking Neural Networks - Emergent Mind https://www.emergentmind.com/topics/reward-modulated-spike-timing-dependent-plasticity-r-stdp

144. Reward-Modulated Spike-Timing-Dependent Plasticity ... https://www.bohrium.com/en/sciencepedia/feynman/keyword/r_stdp

145. Acetylcholine-modulated plasticity in reward-driven navigation (Zannone ... https://modeldb.science/showmodel?model=245018

146. Neuromodulated STDP, and Theory of Three-Factor Learning Rules https://farahbaracat.com/lit_review/2021/06/27/Gerstner/

147. Cholinergic interneuron inhibition potentiates corticostriatal ... https://www.sciencedirect.com/science/article/pii/S2211124722008282

148. Cholinergic control of striatal GABAergic microcircuits - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC9648160/

149. Rust's Zero-Cost Abstractions, What Monomorphization Actually Does to ... https://dev.to/shayan_holakouee/rusts-zero-cost-abstractions-what-monomorphization-actually-does-to-your-code-5dim

150. Why Rust Traits are the Key to Zero-Cost Abstractions You Need to ... https://medium.com/@anshusinghal703/why-rust-traits-are-the-key-to-zero-cost-abstractions-you-need-to-master-5bb2c58e36c7

151. Rust's Multi-Paradigm Design - grasp.study https://paths.grasp.study/modules/82a364d7-cbff-4631-9b0f-cf1a102882c4/lessons/4a728c02-2664-4ffa-86f6-f203b3db3a99

152. Advanced Trait Patterns in Rust: Specialization and Zero-Cost ... https://levelup.gitconnected.com/advanced-trait-patterns-in-rust-specialization-and-zero-cost-abstractions-63f960c55d9f

153. Zero-cost abstractions in Rust | A journey into a wild pointer https://carette.xyz/posts/zero_cost_abstraction/
154. Rust Zero-Cost Abstractions | Claude Code Skill Guide - MCP Market https://mcpmarket.com/tools/skills/rust-zero-cost-abstractions-4
155. [PDF] Meta-Monomorphizing Specializations - arXiv https://arxiv.org/pdf/2602.12973
156. Continual Learning with Neuromorphic Computing - arXiv https://arxiv.org/html/2410.09218v3
157. Neuromorphic AI Inference in Silicon Valley 2026 https://stanfordtechreview.com/articles/neuromorphic-computing-for-energy-efficient-ai-inference-in-silicon-valley-2026
158. Neuromorphic computing for content-based image retrieval - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC8985975/
159. A compiler for mapping PyTorch models to neuromorphic targets. https://eureka.patsnap.com/report-a-compiler-for-mapping-pytorch-models-to-neuromorphic-targets
160. A TTFS-based energy and utilization efficient neuromorphic CNN ... https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2023.1121592/full
161. awesome-neuromorphic-hw/README.md at main - GitHub https://github.com/open-neuromorphic/awesome-neuromorphic-hw/blob/main/README.md
162. Speeding up Transformer CPU inference in Google Cloud - Blog https://blog.x.com/engineering/en_us/topics/insights/2021/speeding-up-transformer-cpu-inference-in-google-cloud
163. De-mystifying PyTorch for ASICs: When (and Why) To ... - YouTube https://www.youtube.com/watch?v=2jweAgKcT8w
164. [PDF] Regional heterogeneity of striatal cholinergic interneurons https://europepmc.org/articles/pmc12971414?pdf=render
165. HASNAS: A Hardware-Aware Spiking Neural Architecture Search ... https://arxiv.org/html/2407.00641v1
166. Impact of Neuron Models on Spiking Neural Network Performance https://pmc.ncbi.nlm.nih.gov/articles/PMC12779659/
167. I scaled a pure Spiking Neural Network (SNN) to 1.088B parameters ... https://www.reddit.com/r/OpenSourceeAI/comments/1sk4953/i_scaled_a_pure_spiking_neural_network_snn_to/
168. Update arXiv papers about Spiking Neural Networks daily. · GitHub https://github.com/SpikingChen/SNN-Daily-Arxiv

169. I built an AI-driven Oracle using Spiking Neural Networks (SNN) in ... https://www.reddit.com/r/defi/comments/1uxbxzs/i_built_an_aidriven_oracle_using_spiking_neural/

170. Implementing Rust like traits for C++ 20 with no runtime overhead https://www.youtube.com/watch?v=VWAtnhOg-0Y

171. SuReLI/RL-STDP: Reinforcement Learning Spike-Timing ... - GitHub https://github.com/SuReLI/RL-STDP

172. How Trait Specialization Works (and Why It's Still Experimental) https://medium.com/@theopinionatedev/how-trait-specialization-works-and-why-its-still-experimental-fe5712636877

173. Rust Traits Explained: How They Work and Why They Matter - Leapcell https://leapcell.io/blog/rust-traits-explained

174. Traits: Defining Shared Behavior | Rust Book - YouTube https://www.youtube.com/watch?v=WDFAOyGmyE0

175. RewardModulatedSTDP in spiking_neural_networks::neuron - Docs.rs https://docs.rs/spiking_neural_networks/latest/spiking_neural_networks/neuron/plasticity/struct.RewardModulatedSTDP.html

176. [PDF] Self-Adaptive Spike-Time- Dependent Plasticity of Metal- Oxide ... https://web.ece.ucsb.edu/~strukov/papers/2016/SciRepSpiking2016.pdf

177. Tricks of the Trait: Enabling Ergonomic Extractors - YouTube https://www.youtube.com/watch?v=7DOYtnCXucw

178. Rust Release Notes https://doc.rust-lang.org/beta/releases.html

179. The Rust ML Ecosystem Is Quietly Becoming Production-Ready CAESpgEB6zswFQykwSWdL4ojrBHCKU1bUHqlsOwPkBXiS-zc1p3gyFrrMQYNejz12i6i_Iv8zC9yX-ciHvq5c9dpE3LaYBQkJFMhqNqOZMHqtWztDCn-pvwHj3gSVvSj0a1I7CMkOofxvyAyVJV5gj8I5g77GibmaF-lIWvzPsMGgmx9qVouOFR6TLGsQWvaYs9vJ4_fVooV8iuErZYtIstLqF8HVgPtGCNg

180. e-tornike/best-of-ml-rust: A ranked list of awesome machine learning ... CAESZAHrOzAV6XNcgdEnQ47Eef65b4VNM5efY84uogECO4o_jQPg3xmQqBem0h12j-8-ABLlyFK5snQ5CYg_4EjOPpy3SPIzT-1EDnWLZHvHgYVxCZUn27nJeD4YzFfdxaWRP-yY5Vw

181. Rust for AI/ML in 2026 — Can It Replace Python? (Burn ... - YouTube CAESZwHrOzAVReQVYkHizfs2lMeN7qBC6ikQW59iXeJuOK1OkLO-vzP5eTb4zlaCrMCI9MDsmse6oifqQj99hZsxCZOuNY2bnTMoptR63eg_dIjQ1Cekh6LaJgg4T6WMvH0uZpggYFFHmMI

182. Rust Meets AI: Why the Future of Machine Learning Runs on Safe ... CAESrQEB6zswFWnw0jkOYb_O4tKiObIwfpinmMNx9mvX9eJVpI4qzWB2SzygiHEyKVp9fGOvPlYt6wOgfBsvLWBR6UfsvEZcbxC9CkawDXxiBeGXoRSAOTkyAmw4Ax5d52exJ2

sdYa5EBeRE2q3AlrCJUWkwlLBmUIEaYtXa87vysyQJSy747VcOQBITz0DTeTlJxikUJSjSITmcqJY1o-2xbknKY9XHnoNo0aehCU3Lpg

183. Neural networks - what crates to use? : r/rust - Reddit https://www.reddit.com/r/rust/comments/aofkg4/neural_networks_what_crates_to_use/
184. fomorians/neural-rs: Spiking neural network library for Rust - GitHub https://github.com/fomorians/neural-rs
185. Neuroscience - Categories - crates.io: Rust Package Registry https://crates.io/categories/science::neuroscience
186. neuromod - crates.io: Rust Package Registry https://crates.io/crates/neuromod/security
187. fast-neural-network - crates.io: Rust Package Registry https://crates.io/crates/fast-neural-network
188. neural-networks - Keywords - crates.io: Rust Package Registry https://crates.io/keywords/neural-networks
189. memory-pool-allocator - crates.io: Rust Package Registry https://crates.io/crates/memory-pool-allocator
190. neurons - crates.io: Rust Package Registry https://crates.io/crates/neurons
191. adriann - crates.io: Rust Package Registry https://crates.io/crates/adriann
192. Rust for Machine Learning: An Introduction https://reintech.io/blog/rust-for-machine-learning-introduction
193. Dopaminergic regulation of hippocampal plasticity, learning ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9911454/
194. Dopamine builds and reveals reward-associated latent ... https://www.nature.com/articles/s41467-024-53976-x
195. Dopamine STDP Synapse - SC-NeuroCore Documentation https://anulum.github.io/sc-neurocore/api/dopamine_stdp/
196. Cholinergic-dependent dopamine signals in mouse dorsal striatum ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12622003/
197. Cholinergic-dependent dopamine signals in mouse dorsal striatum ... https://www.biorxiv.org/content/10.1101/2025.09.30.679538v1
198. Prefrontal Cortex-Driven Dopamine Signals in the Striatum Show ... https://pmc.ncbi.nlm.nih.gov/articles/PMC7511190/
199. Computing of neuromorphic materials: an emerging approach for ... https://pubs.rsc.org/ma/article/4/23/5882/812399/Computing-of-neuromorphic-materials-an-emerging

200. Paper Abstracts – ASPLOS 2024 https://www.asplos-conference.org/asplos2024/main-program/abstracts/index.html

201. Rust for Embedded Systems: Current State and Open ... https://arxiv.org/pdf/2311.05063

202. The Rust Programming Language for Embedded Software ... https://opus4.kobv.de/opus4-haw/frontdoor/deliver/index/docId/786/file/I000819827Thesis.pdf/1000

203. Security in Embedded Devices: RTOS and Linux on ARM https://www.linkedin.com/pulse/security-embedded-devices-rtos-linux-arm-gheorghe-prelipcean-upnlf